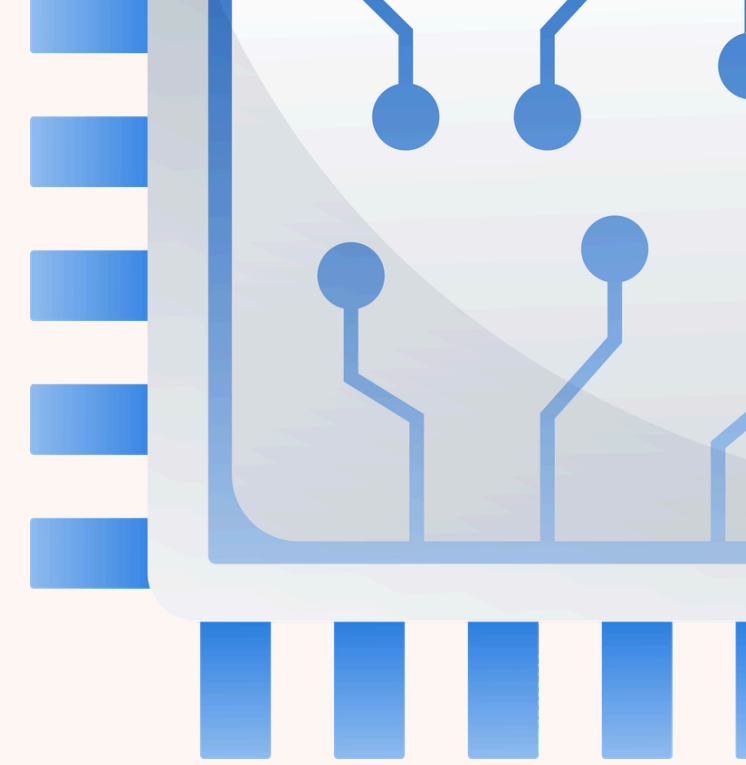
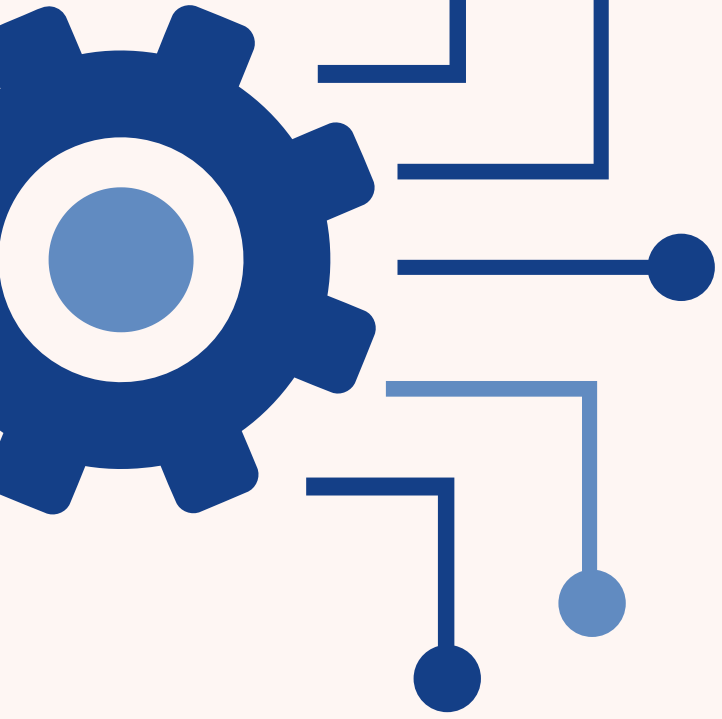
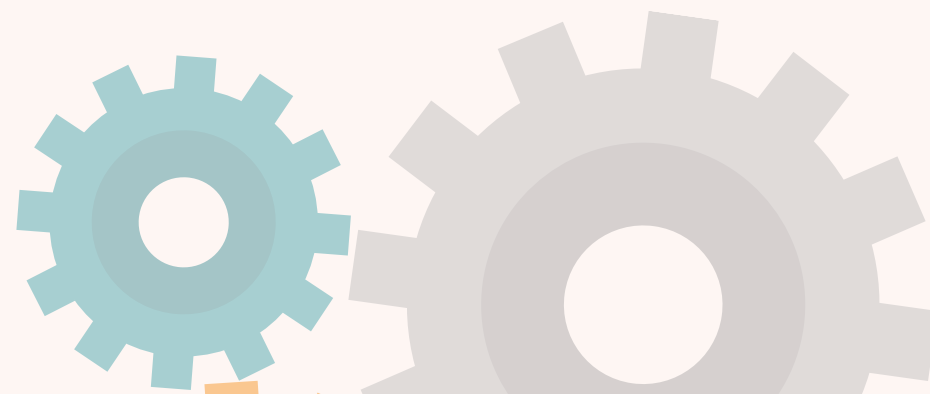
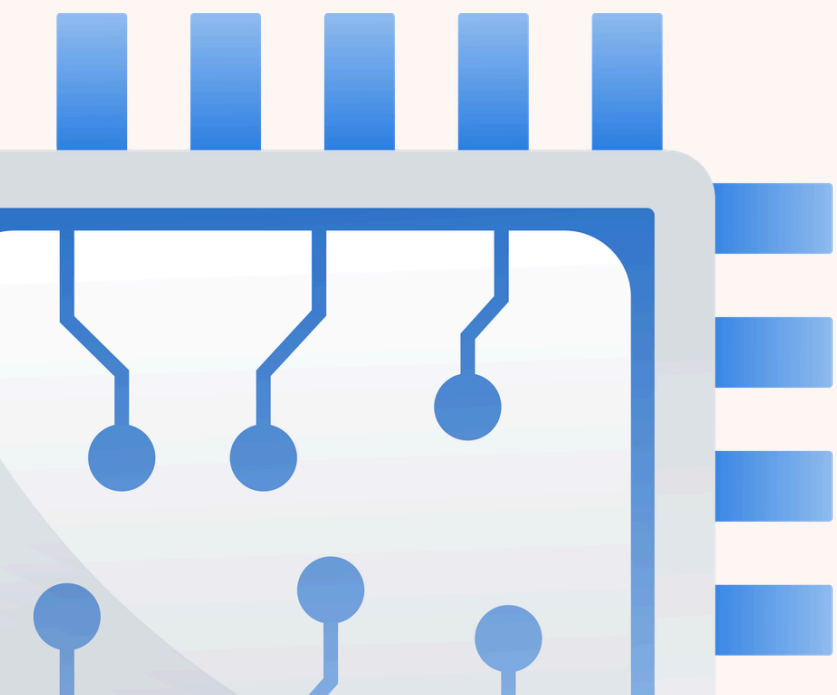




REDES NEURONALES

perceptron xor



EQUIPO

Christian Omar Ceballos Hernández.



EL PROBLEMA DE LA COMPUERTA XOR

A	B	Out
0	0	0
0	1	1
1	0	1
1	1	0

- El dilema matemático: El perceptrón simple es un clasificador lineal. Esto significa que solo puede resolver problemas donde los datos sean linealmente separables (que se puedan dividir con una sola línea recta en 2D).
- El fallo con XOR: Si revisamos la tabla de verdad de la compuerta XOR, los puntos que dan la misma salida están cruzados en diagonal. Es matemáticamente imposible trazar una línea recta que deje los valores verdaderos de un lado y los falsos del otro.



LA SOLUCIÓN (INGENIERÍA DE CARACTERÍSTICAS)

¿Qué es un Kernel?

Para solucionar la no-linealidad, aplicamos ingeniería de características mediante un Kernel. Consiste en tomar los datos originales en dos dimensiones (x_1 , x_2) y proyectarlos matemáticamente a un espacio de mayor dimensión (3D).

La transformación matemática:

Definimos una tercera dimensión (x_3) como el producto de las dos entradas originales:

$$x_3 = x_1 * x_2$$

Nueva Tabla de Datos en 3D:

Al mapear las entradas con la lógica de $[-1, 1]$ que maneja nuestro algoritmo, el espacio se transforma así:

x_1	x_2	$x_3 = x_1 \cdot x_2$	Salida XOR (y)	Coordenada 3D final
-1	-1	1	-1 (Falso)	$A(-1, -1, 1)$
-1	1	-1	1 (Verdadero)	$B(-1, 1, -1)$
1	-1	-1	1 (Verdadero)	$C(1, -1, -1)$
1	1	1	-1 (Falso)	$D(1, 1, 1)$

Explicación:

Nota cómo los puntos con salida verdadera (B y C) se hunden hacia el nivel $z = -1$, mientras que los puntos con salida falsa (A y D) se elevan al nivel $z = 1$. ¡Ahora los datos sí se pueden dividir usando un plano horizontal!



QUE KERNEL UTILIZAMOS?

El kernel que utilizamos es un Kernel Polinomial de Segundo Grado (también conocido como mapeo cuadrático o polinomial homogéneo).

Específicamente, la operación matemática que añade esa tercera dimensión:

$$x_3 = x_1 * x_2$$

recibe el nombre de término de interacción de segundo orden.

¿Por qué se llama así y cómo funciona?

En dos dimensiones (2D), los puntos de la XOR están distribuidos de forma que no puedes cruzarlos con una línea recta. Al aplicar este kernel polinomial, obligas a los datos a multiplicarse entre sí.

Mira lo que pasa matemáticamente con los signos:

- Si las entradas son iguales ($1 * 1$ o $-1 * -1$), el resultado de la multiplicación siempre da 1 positivo (lo que eleva esos puntos en el eje Z).
- Si las entradas son diferentes ($1 * -1$ o $-1 * 1$), el resultado siempre da -1 negativo (lo que hunde esos puntos en el eje Z).



IMPLEMENTACIÓN EN SCI-LAB

Para resolver el problema, desarrollamos un script en Sci-Lab que entrena el perceptrón desde cero empleando las 3 dimensiones. El algoritmo calcula los pesos óptimos (w_1 , w_2 , w_3) y el umbral o bias (b) de forma iterativa mediante la regla de aprendizaje del perceptrón.

```
6
7 clear;
8 clc;
9
10 printf("=== PROGRAMA: PERCEPTRON XOR EN 3D ===\n\n");
11
12 //1. Entradas originales de la compuerta XOR (usando -1 y 1)
13 x1 = [-1; -1; -1; -1];
14 x2 = [-1; -1; -1; -1];
15
16 //2. INGENIERÍA DE CARACTERÍSTICAS (Kernel polinomial 3D)
17 //Creamos una tercera dimensión multiplicando x1 por x2
18 x3 = x1 .* x2;
19
20 //3. Salidas esperadas de la compuerta XOR
21 //Si son iguales da -1 (Falso), si son diferentes da 1 (Verdadero)
22 y = [-1; -1; -1; -1];
23
24 //Mostrar los datos organizados en el espacio 3D
25 printf("Datos de entrenamiento [x1, x2, x3] -> Salida XOR:\n");
26 disp([x1, x2, x3, y]);
27
28 //4. Inicialización de los Pesos y el Umbral (Bias)
29 w1 = 0.0;
30 w2 = 0.0;
31 w3 = 0.0;
32 b = 0.0;
33
34 lr = 0.5; //Tasa de aprendizaje
35 max_iter = 50; //Límite de iteraciones para evitar bucles infinitos
36
37 printf("\n=== Iniciando Entrenamiento ===\n");
```

```
38
39 for epoch = 1:max_iter
40     errores = 0;
41     for i = 1:4
42         //Calcular la combinación lineal
43         suma_lineal = (w1 * x1(i)) + (w2 * x2(i)) + (w3 * x3(i)) + b;
44         //Función de activación (Signo: regresa -1 si es negativo, 1 si es positivo)
45         if suma_lineal >= 0 then
46             prediccion = 1;
47         else
48             prediccion = -1;
49         end
50         //Calcular el error de la predicción
51         error_actual = y(i) - prediccion;
52         //Si hay error, actualizamos los pesos de forma uniforme
53         if error_actual <> 0 then
54             w1 = w1 + lr * error_actual * x1(i);
55             w2 = w2 + lr * error_actual * x2(i);
56             w3 = w3 + lr * error_actual * x3(i);
57             b = b + lr * error_actual;
58             errores = errores + 1;
59         end
60     end
61     //Si terminamos una vuelta por los 4 puntos sin errores, ya terminó
62     if errores == 0 then
63         printf("¡*! Convergencia alcanzada con éxito en la iteración %d!\n", epoch);
64         break;
65     end
66 end
```

RESULTADOS DEL ENTRENAMIENTO

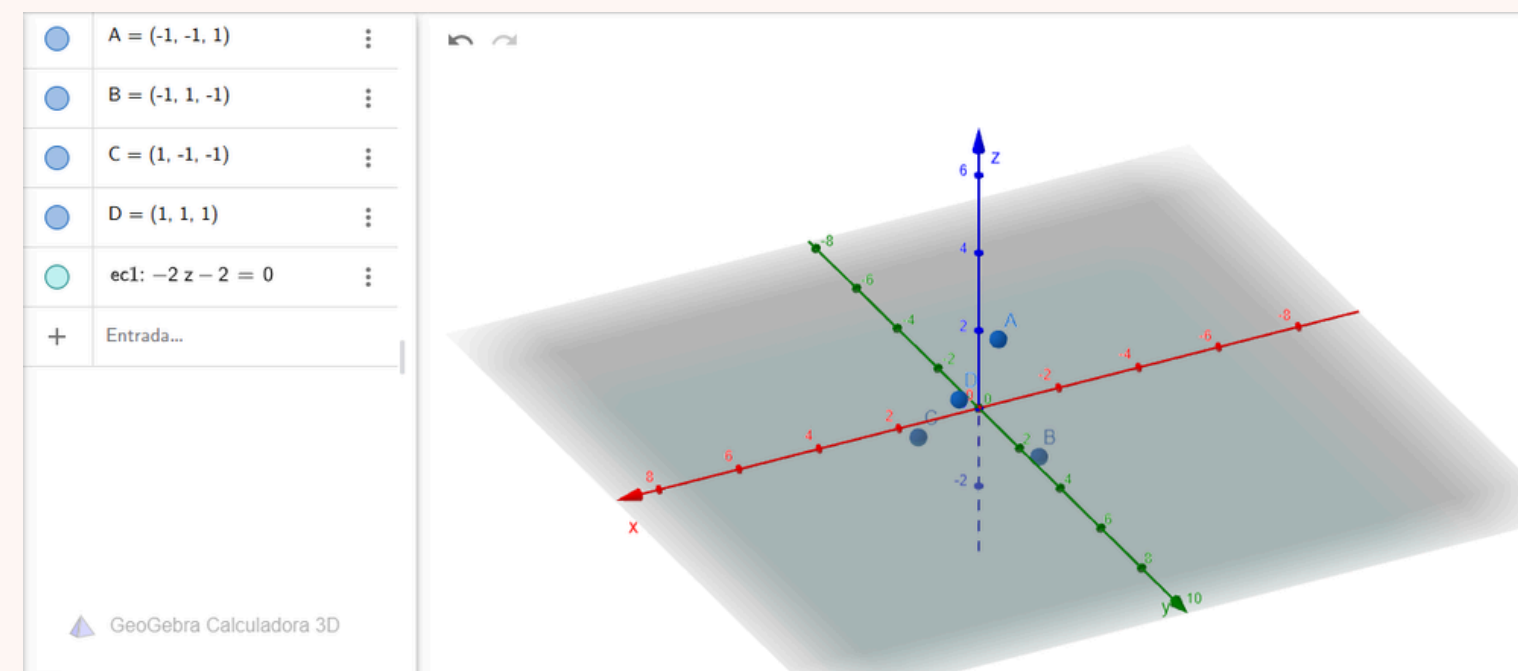
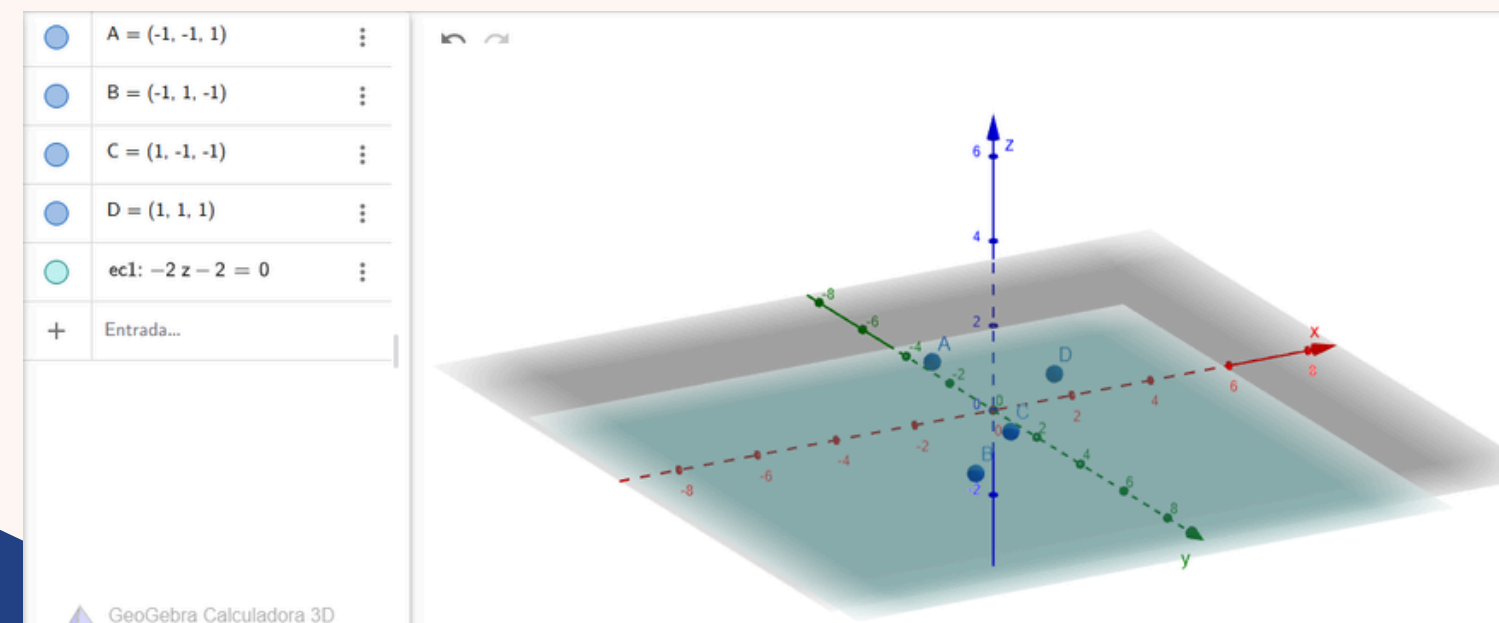
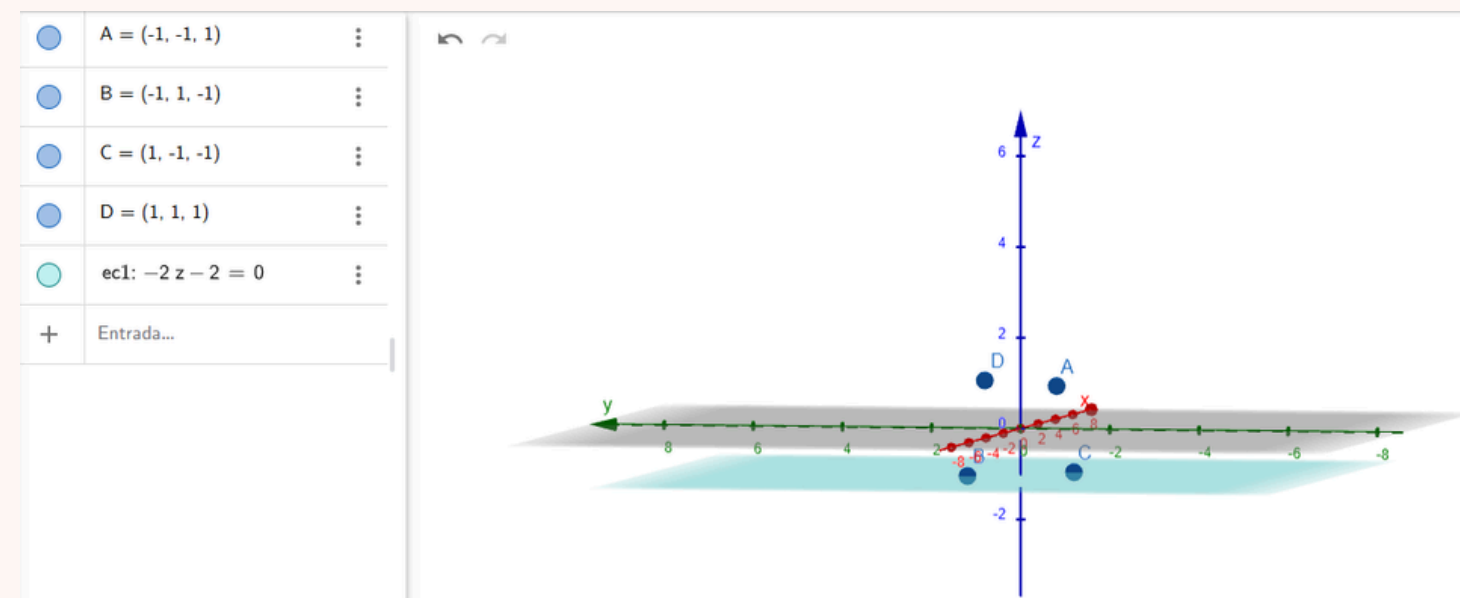
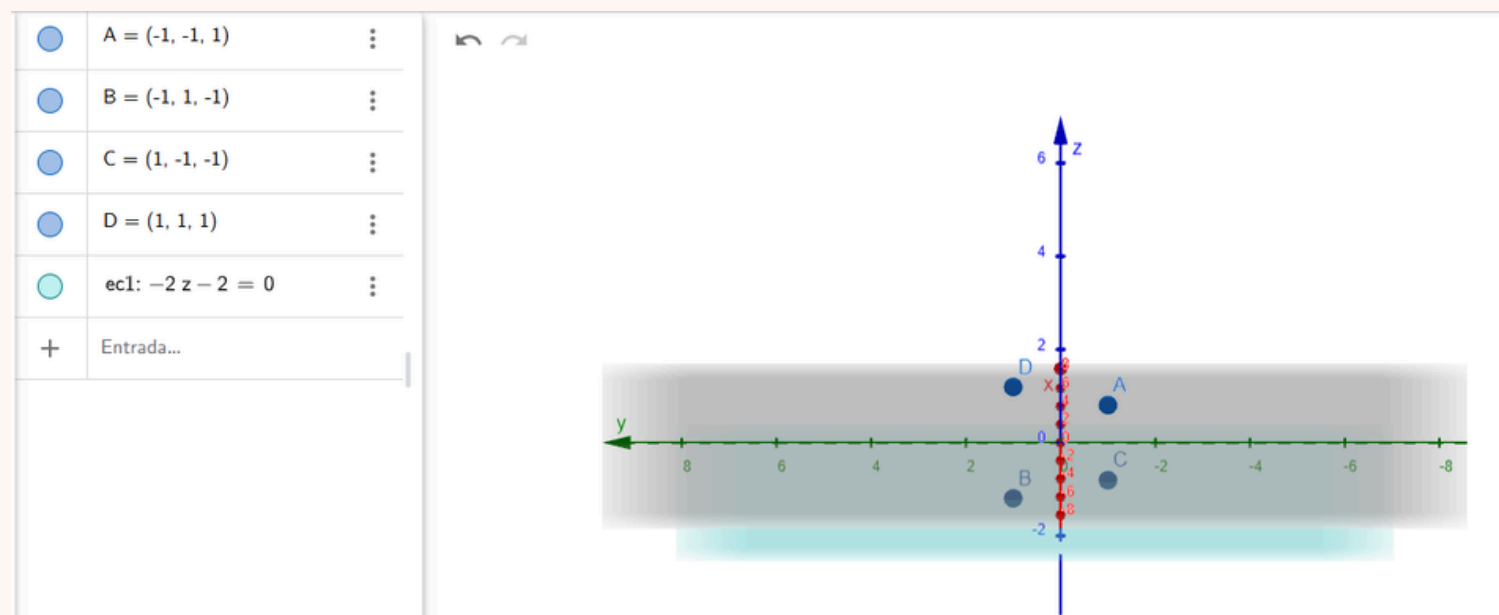
Al ejecutar el algoritmo en Sci-Lab, el modelo converge rápidamente y encuentra los siguientes parámetros óptimos:

- Peso w_1 (x_1): 0
- Peso w_2 (x_2): 0
- Peso w_3 (x_3): -2
- Umbral / Bias (b): -2

La Ecuación del Hiperplano Separador
Sustituyendo los pesos en la ecuación general del plano ($w_1x + w_2y + w_3z + b = 0$), obtenemos la ecuación final que introducimos en GeoGebra:

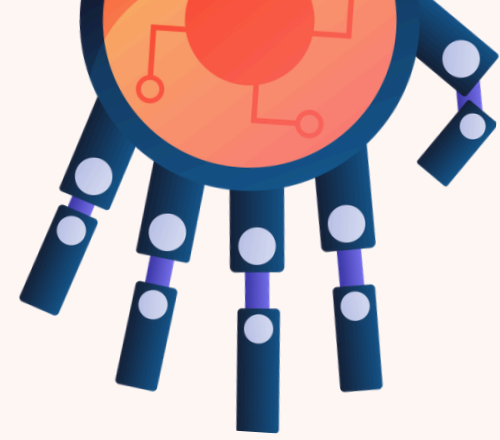
$$-2z - 2 = 0$$

RESULTADOS DE LA SIMULACIÓN VISUAL (GEOGEBRA 3D)



CONCLUSION

- La ingeniería de características (Kernel) es una herramienta fundamental en el aprendizaje automático que permite a modelos simples, como el perceptrón, resolver problemas complejos y no lineales sin necesidad de cambiar la estructura interna del algoritmo.
- Al elevar los datos de la compuerta XOR a una tercera dimensión mediante el producto de sus variables, lo que antes era imposible en 2D se volvió completamente separable mediante un hiperplano en 3D, demostrando la efectividad matemática del método.



**MUCHAS
GRACIAS**

